

RxTrack

Pharmacy Inventory Management — Market Research & Prototype Report

Candidate Assessment: Hospital Industry Product Discovery & Prototype Development

Prepared for: MedAura, Calicut

Prepared by: Krishnakishor R

Contents

1. Methodology
2. Problem Statement
3. Target Users
4. Existing Solutions in the Market
5. Gap Analysis
6. Proposed Solution
7. Assumptions & Scope
8. System Architecture
9. Prototype Features
10. Local Setup Instructions
11. Future Enhancements
12. Conclusion

1. Methodology

This report follows the structure requested in the MedAura candidate assessment brief: identify a real healthcare problem, research the existing market, identify gaps, propose a solution, and build a working prototype. The research approach combined:

- **Secondary research** — reviewing publicly available product pages, feature listings, and documentation for established pharmacy/hospital inventory software used in the Indian market.
- **Comparative analysis** — evaluating at least three existing solutions against a consistent set of criteria (scope, target user, core features, complexity) to surface a genuine gap rather than an assumed one.
- **Scoped prototyping** — translating the identified gap into a minimal, working full-stack application (not a mockup) so the proposed solution could be evaluated hands-on, not just described.

The prototype was built with a React frontend and a Django REST Framework backend, as specified in the assessment's clarified requirements, using SQLite as the local database.

2. Problem Statement

Hospital and clinic pharmacies routinely lose track of two things that matter most for patient safety and cost control: how much stock they actually have, and how close that stock is to expiring. Manual registers or spreadsheets make it easy to miss a medicine crossing its reorder threshold, or dispense from a batch that is about to expire instead of one further out. Existing commercial pharmacy software tends to solve this only as a side-effect of a much larger billing/GST/POS system, which is more than many small hospital pharmacies or single departments need or can justify adopting.

3. Target Users

- **Pharmacy in-charge / pharmacist** — needs a fast way to check stock levels, add new stock, and see what needs reordering or is close to expiry, without navigating a full ERP.
- **Hospital procurement staff** — needs visibility into low-stock and expiring items to plan purchase orders and supplier follow-ups.
- **Department / ward in-charge (smaller setups)** — needs a lightweight tool to track their own medicine stock without depending on a hospital-wide system built for the main pharmacy.

4. Existing Solutions in the Market

Three established players were reviewed to understand how the current market approaches pharmacy inventory, and where their scope and complexity sit relative to a single-department tool.

Solution	Positioning	Core Capabilities	Complexity for a Single Pharmacy
Marg ERP 9+	India's most established retail/distribution pharmacy ERP, used across 9 lakh+ users.	Inventory management, POS, sales force automation, multi-location inventory, batch-wise stock tracking, 1000+ report types.	High — a full retail/distribution ERP; pharmacy inventory is one module among many.
Medical365	Pharmacy module bundled inside a larger Hospital Management System (HMS).	Multi-store management (Main Store / OPD / IPD Pharmacy), granular GST reporting, FEFO dispensing with 30/60/90-day expiry alerts.	High — positioned as an add-on to a full HMS, not a standalone tool.
Mediyaam	Cloud-native pharmacy ERP for retail, hospital, and wholesale pharmacies.	Unified POS billing, batch-wise inventory, GST compliance, procurement, analytics.	Medium-High — broad ERP feature set aimed at full pharmacy operations, not a single department.

5. Gap Analysis

All three solutions are commercial-grade ERPs that bundle billing, GST compliance, POS, and multi-branch management alongside inventory tracking. That breadth is valuable for a full pharmacy operation, but it creates real friction for a smaller use case:

- **Adoption overhead** — onboarding a full ERP (accounts, GST setup, POS hardware/software integration) is disproportionate effort for a team that only wants to know what's in stock and what's expiring.
- **Cost** — commercial ERP pricing is typically structured around full pharmacy/retail operations, not a single ward or department's inventory.
- **Focus** — stock and expiry visibility, the highest-frequency daily task, is one screen among many in these systems rather than the primary interface.

This leaves a gap for a lightweight, focused inventory tracker aimed at a single hospital department or small clinic pharmacy — one that surfaces stock and expiry risk clearly, without requiring adoption of billing or compliance modules it doesn't need.

6. Proposed Solution

RxTrack — a focused pharmacy inventory prototype that covers exactly the recurring daily task: knowing what medicine stock exists, what's running low, and what's approaching expiry, with the ability to add, edit, search, and filter medicines directly.

- A **dashboard** that immediately surfaces what needs attention: total items, stock value, low-stock count, and expired/expiring count, backed by ranked lists of the specific medicines at risk.

- Full **CRUD** on medicine records: name, category, batch number, stock quantity, reorder level, unit price, supplier, and expiry date.
- **Search and filtering** by name, batch number, supplier, category, and stock/expiry status.
- A **shelf-life indicator** per medicine — a visual bar showing days remaining until expiry, so risk is visible at a glance rather than buried in a date column.
- Deliberately **excludes** billing, POS, and GST — out of scope by design, to stay lightweight and fast to adopt for the target users above.

7. Assumptions & Scope

- Single-store / single-department scope — no multi-branch inventory in this prototype.
- No real billing, POS, or GST integration — intentionally out of scope.
- Data is dummy/seeded (30 sample medicines) for demonstration — not connected to a live hospital system.
- Single-user access in the prototype — no role-based login (noted as a future enhancement).
- Currency shown in INR (₹), consistent with the target market.

8. System Architecture

RxTrack follows a standard three-tier architecture: a React single-page frontend, a Django REST Framework API, and a SQLite database, as specified in the assessment's technical requirements.

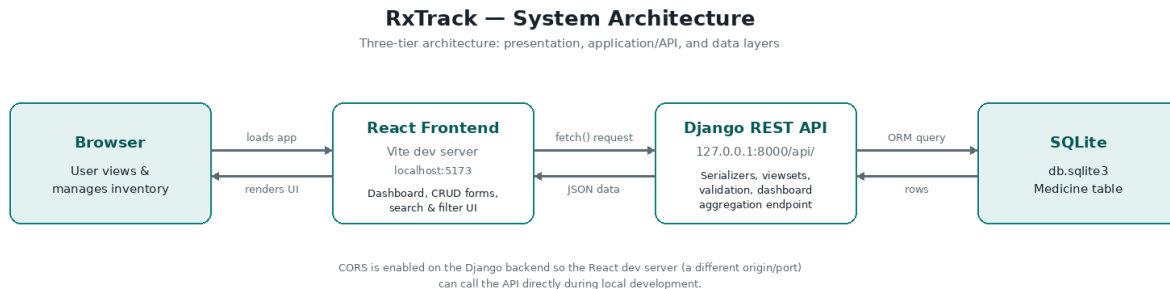


Figure 1 — RxTrack request/response flow across the three tiers.

8.1 Component Breakdown

Layer	Technology	Responsibility
Presentation	React (Vite)	Dashboard, inventory table, add/edit forms with client-side validation, search & filter controls, responsive layout.
Application / API	Django + Django REST Framework	Medicine model, serializer-level validation, REST endpoints (list/create/update/delete/search/filter), dashboard aggregation endpoint.
Data	SQLite	Single-file relational database storing the Medicine table; no separate database server to install or configure.

8.2 Data Model

The core entity is **Medicine**: name, category, batch number (unique), stock quantity, reorder level, unit price, supplier, and expiry date. Status (in-stock / low-stock / expiring-soon / expired) and days-to-expiry are computed server-side at read time rather than stored, so they are always accurate relative to the current date.

8.3 API Endpoints

Method	Endpoint	Description
GET	/api/medicines/	List medicines. Supports ?search=, ?category=, ?status=
POST	/api/medicines/	Create a medicine (validated server-side)
GET	/api/medicines/{id}/	Retrieve one medicine
PUT	/api/medicines/{id}/	Update a medicine
DELETE	/api/medicines/{id}/	Delete a medicine

Method	Endpoint	Description
GET	/api/medicines/dashboard/	Aggregated dashboard statistics

9. Prototype Features

Mapped against the assessment's prototype requirements:

Requirement	Implementation
Responsive UI	Sidebar layout collapses to a stacked, touch-friendly layout below 720px width.
CRUD Operations	Full create/read/update/delete on medicines via the Django REST API.
Search & Filtering	Text search (name/batch/supplier) plus category and status filters, combinable.
Dashboard	Stock value, item counts, low-stock and expiring-soon lists, computed live from data.
Form Validation	Client-side checks (required fields, non-negative numbers) plus server-side serializer validation as the source of truth.
Dummy Data	A seed command generates 30 medicines with a realistic spread of stock levels and expiry dates.
Well-Structured Code	Separated Django apps (models/serializers/views/urls) and componentised React (Sidebar, Dashboard, Inventory, MedicineForm, ExpiryBar).

10. Local Setup Instructions

Requires Python 3.10+ and Node.js 18+ installed locally.

10.1 Backend (Django API)

```
cd backend
pip install -r requirements.txt
python manage.py migrate
python manage.py seed_data
python manage.py runserver
```

Runs the API at <http://127.0.0.1:8000/api/>

10.2 Frontend (React)

```
cd frontend
npm install
npm run dev
```

Runs the app at <http://localhost:5173>, which talks to the Django API automatically.

10.3 Repository

Full source code, commit history, and this report are available in the project's GitHub repository (link provided in the final submission). The repository README documents the same setup steps, project structure, and API reference in full.

11. Future Enhancements

- Role-based login (pharmacist vs. admin) instead of open access.
- Batch-level FEFO (First-Expiry-First-Out) dispensing logic.
- Automatic purchase order generation when stock crosses the reorder level.
- Multi-store / multi-branch inventory support.
- Barcode scanning for faster stock entry.
- Audit log of stock changes over time.

12. Conclusion

The pharmacy inventory space is well served for full-scale ERP needs, but underserved for teams that just need clear, low-friction visibility into stock and expiry risk. RxTrack targets that specific gap with a working prototype: a React frontend, a Django REST API, and a SQLite database, covering the full set of CRUD, search, filtering, dashboard, and validation requirements set out in the assessment brief.